Introduction to the Link Layer

The link layer is an essential part of computer networks, responsible for moving data across physical communication channels. To understand it clearly, some key terms are introduced:

- **Nodes:** Any device that runs a link-layer (Layer 2) protocol is called a node. Examples include hosts, routers, switches, and WiFi access points.

- **Links:** The communication channels that connect adjacent nodes are called links.

For a datagram (a network-layer packet) to travel from a **source host** to a **destination host**, it must pass through a sequence of links that form the end-to-end path. At each step, the node transmits the datagram across its local link.

Example: Six-Link Path

In a company network, sending a datagram from a wireless host to a server involves multiple links:

1. **WiFi link** between the wireless host and the WiFi access point.

2. **Ethernet link** between the access point and a link-layer switch.

3. **Link** between the link-layer switch and a router.

4. **Link** between two routers.

5. **Ethernet link** between the router and another link-layer switch.

6. **Ethernet link** between that switch and the server.

At each of these hops, the transmitting node encapsulates the datagram into a **link-layer frame** before sending it across the link.

Transportation Analogy

The link layer's function can be compared to planning a tourist's journey:

- The **tourist** represents the datagram.

- The **travel agent** represents the routing protocol, which decides the full end-to-end path.
- Each **segment of the trip** (limousine, plane, train) corresponds to a **link** between adjacent locations.
- Each **transportation mode** (limousine company, airline, train service) corresponds to a **link-layer protocol**, responsible only for its own part of the journey.

Even though the transportation modes are different, they all provide the same fundamental service: **moving the traveler from one location to the next adjacent one**. Likewise, link-layer protocols may differ (WiFi, Ethernet, etc.), but they all perform the basic task of moving data across a single link.

In summary, the link layer ensures that datagrams are transmitted hop by hop across different types of links until they finally reach their destination, just as a tourist uses different modes of transportation to complete a long trip.

6.1.1 The Services Provided by the Link Layer

The main purpose of the link layer is to move a **datagram** from one node to an adjacent node over a single link. However, the exact services vary depending on the specific link-layer protocol. Key services include:

1. Framing

- Almost all link-layer protocols encapsulate network-layer datagrams into **frames** before transmission.
- A frame consists of:
    - A **data field**, which holds the network-layer datagram.
    - **Header fields**, whose structure depends on the link-layer protocol.
- Different protocols define different frame formats, which will be studied later.

2. Link Access (Medium Access Control – MAC)

- Determines how frames are placed onto the link.

- **Point-to-Point Links:** Simple case with one sender and one receiver; sender can transmit whenever the link is idle (MAC may be minimal or unnecessary).

- **Broadcast Links (Shared Medium):** Multiple nodes share the same link, leading to the **multiple access problem**.

    - A **MAC protocol** coordinates transmissions so that frames do not collide and communication remains efficient.

3. Reliable Delivery

- Ensures that each datagram is delivered across the link **without errors**.

- Achieved using **acknowledgments** and **retransmissions**, similar to TCP in the transport layer.

- **When used:**

    - Important for **high-error links** (e.g., wireless), where correcting errors locally is more efficient than forcing end-to-end retransmissions.

- **When avoided:**

    - Considered unnecessary overhead for **low-error links** (fiber optics, coaxial, twisted-pair copper), so many wired protocols skip it.

4. Error Detection and Correction

- Physical issues (signal attenuation, electromagnetic noise) can flip bits ($0 \leftrightarrow 1$).

- To prevent forwarding corrupted data:

    - Transmitter adds **error-detection bits** to the frame.

    - Receiver checks these bits to detect errors.

- Compared to the Internet checksum used in transport and network layers, **link-layer error detection is more advanced and implemented in hardware** for speed.

- **Error correction** goes further:
    - Receiver identifies exactly where errors occurred in the frame.
    - Receiver corrects the bits automatically.

👉 In summary, link-layer protocols not only provide the basic function of moving data between adjacent nodes but may also add features like **framing, access control, reliability, and error handling** to ensure smooth and accurate communication across diverse physical links.

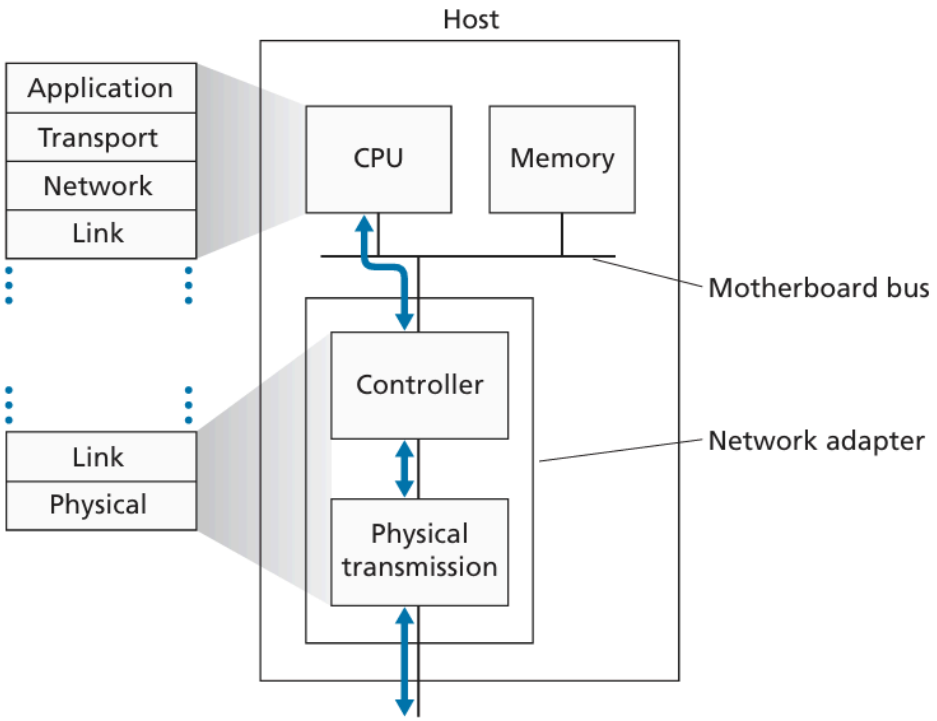


Figure 6.2 ♦ Network adapter: Its relationship to other host components and to protocol stack functionality

6.1.2 Where Is the Link Layer Implemented?

The link layer is responsible for connecting the software world of protocols with the physical transmission of data. Its implementation is distributed across **hardware and software**, often

with the hardware doing most of the low-level work while the software manages control and higher-level tasks.

Network Adapter (NIC)

- The **network adapter**, also known as a **network interface controller (NIC)**, is the primary hardware component for the link layer.
- It may be:
    - Integrated into the **motherboard chipset**, or
    - Implemented as a **dedicated Ethernet/WiFi chip**.
- The NIC performs most link-layer functions in **hardware**, such as:
    - **Framing** (encapsulating datagrams into link-layer frames).
    - **Link access** (following MAC rules).
    - **Error detection** (setting and checking error bits).
- Examples:
    - Intel's **700 series adapters** implement Ethernet.
    - Atheros **AR5006** controller implements WiFi (802.11).

Sending Side Operations

1. Higher-layer protocols (e.g., network layer) create a **datagram** in host memory.
2. The controller (NIC) takes this datagram and **encapsulates it into a frame**.
3. It fills in required frame fields (headers, error-detection bits).
4. The controller then **transmits the frame** onto the link, following the MAC protocol.

Receiving Side Operations

1. The NIC **receives the frame** from the link.

2. It extracts the **network-layer datagram** from the frame.
3. If error detection is enabled:
    - The sender sets error-detection bits.
    - The receiver verifies these bits and detects errors.
4. The NIC then **passes the datagram up** to the network layer for further processing.

Role of Software in the Link Layer

While most link-layer functions are in hardware, some tasks are implemented in software on the host's **CPU**:

- Assembling **link-layer addressing information**.
- Activating and controlling the NIC hardware.
- Responding to **interrupts** from the NIC (e.g., when frames are received).
- Handling **error conditions**.
- Passing the datagram to the **network layer**.

Key Point

The **link layer is a hybrid of hardware and software**.

- **Hardware (NIC):** Handles fast, repetitive, low-level tasks (framing, transmission, error detection).
- **Software (host CPU):** Manages higher-level control, addressing, and interactions with the OS and protocol stack.

Thus, the link layer is the **meeting point of software and hardware** in the network protocol stack, ensuring that datagrams can be reliably moved from memory to the physical communication link and vice versa.

6.2 Error-Detection and -Correction Techniques

The link layer often provides services to detect and correct **bit-level errors** that occur when frames are transmitted between physically connected nodes. These errors can happen due to noise, signal attenuation, or interference on the communication channel. While error control also appears at the **transport layer** (e.g., TCP checksums), here the focus is on mechanisms used at the link layer.

General Process (Figure 6.3)

1. **Sender Side**
    - Data (**D**) to be transmitted is combined with **error-detection and correction bits (EDC)**.
    - The protected information usually includes not only the **datagram** from the network layer but also link-layer headers (e.g., addressing info, sequence numbers).
    - Together, **D + EDC** are transmitted across the link.
2. **Transmission**
    - The frame passes through a **bit error-prone link**, where some bits in **D** or **EDC** may flip.
3. **Receiver Side**
    - Receives **D′ and EDC′**, which may differ from the original values.
    - The receiver checks whether **all bits in D′ are correct**, using the error-detection scheme.
    - If bits match expectations → no error detected.
    - If mismatch occurs → error is detected.

Important Points

- **Detection vs. Occurrence:**
    - The receiver's task is to decide if an **error is detected**, not to know with certainty if an error **actually occurred**.
    - Some errors may remain **undetected** (false negatives).
- **Risk of Undetected Errors:**
    - The receiver may accidentally accept corrupted data and pass it to the **network layer**, or misinterpret corrupted **header fields**.
    - Hence, the chosen technique should make the probability of undetected errors **very small**.
- **Trade-off:**
    - **More sophisticated techniques** (stronger error detection/correction, lower probability of undetected errors) require:
        - More computational power.
        - More overhead bits (larger EDC).
    - Simpler techniques add less overhead but may allow more errors to slip through.

Techniques for Error Detection and Correction

The text introduces three important methods:

1. **Parity Checks**
    - Simplest form of error detection.
    - Demonstrates the fundamental idea of checking bit consistency.
    - Can also be extended to correct single-bit errors.
2. **Checksumming Methods**

    - Commonly used at the **transport layer**.
    - Involves summing data segments and appending the result for error verification.
3. **Cyclic Redundancy Checks (CRC)**
    - Widely used in the **link layer** (especially in network adapters).
    - Provides strong error detection capability.
    - Based on polynomial division of data bits.

👉 In summary, the link layer uses error-detection and correction techniques to **increase reliability** over noisy links. The effectiveness of these methods depends on the balance between **error coverage strength** and **efficiency (overhead and computation cost)**.

6.2.1 Parity Checks

Parity checking is one of the simplest forms of error detection, where additional **parity bits** are added to transmitted data to verify accuracy.

Single-Bit Parity (Figure 6.4)

- Suppose the data **D** has *d* bits.
- An **extra parity bit** is added so that the total number of 1s in the entire block (**d + 1 bits**) follows a parity rule:
    - **Even parity:** Total number of 1s must be even.
    - **Odd parity:** Total number of 1s must be odd.
- **Example (Even Parity):**
    - Data: `01110001101011`
    - Number of 1s = 8 (already even).

    - Parity bit = 1 → ensuring overall even count.

Receiver Operation:

- The receiver counts the number of 1s in the received block (data + parity).
- If the count does not match the expected rule (even/odd), at least one bit error is detected.
- However:
    - A single-bit error → always detected.
    - Multiple-bit errors (even number of flipped bits) → **may go undetected**.

Limitations:

- Works only if single errors are common and multiple errors are rare.
- In reality, bit errors often occur in **bursts**, making single parity unreliable.
- In burst conditions, undetected error probability can approach **50%**.

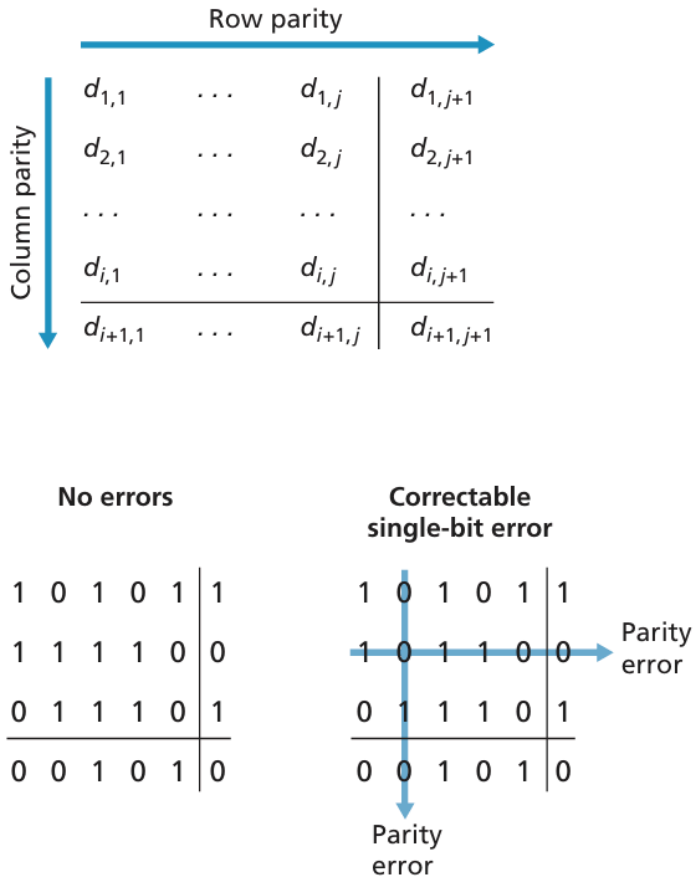


Figure 6.5 ♦ Two-dimensional even parity

Two-Dimensional Parity (Figure 6.5)

To improve detection and enable correction, parity is extended to two dimensions.

- Data bits are arranged in a grid of **i rows** and **j columns**.
- Parity is computed for each row and each column.
- Total parity bits = **i + j + 1**.

How It Works:

1. If a **single-bit error** occurs:
    - The affected **row parity** and **column parity** both show errors.

    - The intersection of the row and column pinpoints the **exact bit** that flipped.
    - The receiver can **correct the error** automatically.
2. If a **parity bit itself** is corrupted:
    - Still detectable and correctable using the same logic.
3. If **two bits** are corrupted:
    - Detected (mismatch in parity counts).
    - But **cannot be corrected**.

Forward Error Correction (FEC)

The ability of the receiver to **both detect and correct errors** is called **Forward Error Correction (FEC)**.

- **Advantages:**
    - Reduces retransmissions from the sender.
    - Allows **immediate correction** at the receiver without waiting for a retransmit request.
    - Important for:
        - **Real-time applications** (e.g., video conferencing, VoIP).
        - **Long-delay links** (e.g., deep-space communications).
- **Practical Use:**
    - Widely used in **audio CDs** and **multimedia systems**.
    - Also combined with **Automatic Repeat reQuest (ARQ)** in networking for stronger error handling.

👉 In summary, **single-bit parity** is simple but weak, while **two-dimensional parity** enhances reliability by enabling both detection and correction of single-bit errors. This forms the foundation of **FEC**, which is crucial in systems requiring minimal retransmissions and real-time error correction.

6.2.2 Checksumming Methods

Checksumming is another error-detection technique where the data bits are grouped into fixed-size integers, summed, and the result is used as error-detection information. This approach is widely used in the Internet, particularly in the **Internet checksum**.

Basic Idea

- The **d bits of data** are divided into a sequence of **k-bit integers**.
- These integers are summed together.
- The resulting sum is used as the **error-detection bits (checksum)**.

The Internet Checksum

- In TCP, UDP, and IP, the Internet checksum is based on this method:
    - Data is divided into **16-bit integers**.
    - These 16-bit values are **summed**.
    - The **1's complement** of this sum forms the checksum.
- **Receiver Verification:**
    - Receiver adds all received 16-bit words, including the checksum.
    - If the result (1's complement) is **all 0s**, then no error is detected.
    - If any bit is 1, an error is flagged.

- **Details:**
    - In **TCP/UDP:** The checksum is calculated over both the **header and the data**.
    - In **IP:** The checksum is calculated only over the **IP header**, because the segment itself (UDP/TCP) already contains its own checksum.
    - In some protocols (e.g., XTP), two checksums are used—one for the header and another for the entire packet.

Advantages of Checksumming

- Requires **very little overhead** (e.g., TCP/UDP checksum uses only **16 bits**).
- Simple to compute, which makes it **fast in software implementations**.

Limitations

- Provides **weaker error protection** compared to **Cyclic Redundancy Check (CRC)**.
- Multiple errors may go undetected if they cancel each other out in the sum.

Why Checksums in Transport Layer, CRC in Link Layer?

- **Transport Layer (TCP/UDP/IP):**
    - Usually implemented in **software** within the host operating system.
    - Needs a **fast and simple** error-detection method.
    - Hence, **checksumming** is preferred.
- **Link Layer:**
    - Implemented in **dedicated hardware** (network adapters).

- Hardware can perform more **complex and powerful** error-detection methods efficiently.
- Hence, **CRC** (Cyclic Redundancy Check) is commonly used.

👉 In summary, checksumming is a lightweight error-detection scheme suitable for **software-based transport layer protocols** like TCP, UDP, and IP. While simple and efficient, it is less robust than CRC, which is used in the **link layer** where hardware can handle the complexity.

Cyclic Redundancy Check (CRC)

Cyclic Redundancy Check (CRC) is a powerful and widely used **error-detection technique** in computer networks. It is also called a **polynomial code**, because the transmitted bit string can be treated as a polynomial, where each bit (0 or 1) is a coefficient. Operations on these bit strings are interpreted as **polynomial arithmetic**.

How CRC Works

1. Let the data to be transmitted be a **d-bit string, D**.
2. Sender and receiver agree on an **(r + 1)-bit generator pattern G**, where the most significant bit (MSB) of G must be **1**.
3. The sender calculates an **r-bit sequence, R**, and appends it to D.
    - This creates a new bit sequence of length **d + r**.
    - The combined sequence is chosen so that it is **exactly divisible by G** (no remainder) using **modulo-2 arithmetic**.
4. At the receiver:
    - The received **(d + r) bits** are divided by G.
    - If the remainder = 0 → no error is detected.

- If the remainder ≠ 0 → an error is detected.

Modulo-2 Arithmetic Rules

- All CRC operations are performed in modulo-2 arithmetic.
- **Addition and subtraction** are identical, both equivalent to **bitwise XOR**.
- Examples:
  - `1011 XOR 0101 = 1110`
  - `1001 XOR 1101 = 0100`
- Multiplication and division follow base-2 arithmetic rules, except that addition/subtraction steps are done using XOR.
- **Multiplication by 2^k** = shifting the bit sequence **left by k places**.

👉 In short, CRC ensures that transmitted data plus CRC bits form a pattern divisible by a known generator polynomial. At the receiver, division by the same generator allows detection of errors. Its use of **polynomial arithmetic with modulo-2 operations** makes it efficient in hardware and highly reliable for detecting burst errors in real networks.

Cyclic Redundancy Check (CRC) Explained with Example

Let's carefully go step by step through how **CRC** works, using a concrete example.

Step 1: Setup

- Suppose the data (**D**) we want to send is:
  $D=101110 (d=6 \text{ bits})$

- Let the generator (**G**) be:
  $G=1001 (r+1=4 \text{ bits, so } r=3) G = 1001 \quad (r+1 = 4 \text{ bits, so } r = 3)$

The most significant bit of G is **1**, as required.

Step 2: Append r Zeros

Before computing the CRC bits (**R**), append **r = 3 zeros** to D:

$D \cdot 2^r = 101110000 D \cdot 2^r = 101110000$

This gives us a temporary 9-bit sequence.

Step 3: Divide by G (Modulo-2 Division)

Now, divide `101110000` by `1001` using **modulo-2 arithmetic (XOR instead of subtraction)**.

Division Steps:

1. `1011` (from the leftmost 4 bits) ÷ `1001` → XOR → result = `010`
    - Bring down next bit → `0101`
2. `0101` ÷ `1001` → can't divide because leading bit is 0.
    - Just bring down next bit → `1011`
3. `1011` ÷ `1001` → XOR → result = `010`
    - Bring down next bit → `1000`
4. `1000` ÷ `1001` → XOR → result = `001`
    - Bring down last bit → `011`

So the **remainder R = 011**.

Step 4: Form the Transmitted Message

Now append the remainder **R** to the original data **D**:

Transmitted bits=D+R=101110011\text{Transmitted bits} = D + R = 101110011

This 9-bit sequence is guaranteed to be divisible by **G = 1001**.

Step 5: Receiver Check

- Receiver gets `101110011`.
- Divides it by `1001`.
- Since it was constructed to be divisible by G, the remainder = **0** if no error occurred.
- If any bits flipped during transmission, the remainder will be **nonzero**, and an error is detected.

✅ **Key Insight:**

- The sender computes remainder **R** and appends it to the data.
- The receiver divides by the same generator **G**.
- If remainder ≠ 0 → **error detected**.

⚡ **Summary with Example:**

- Data: `101110`
- Generator: `1001`
- Remainder: `011`

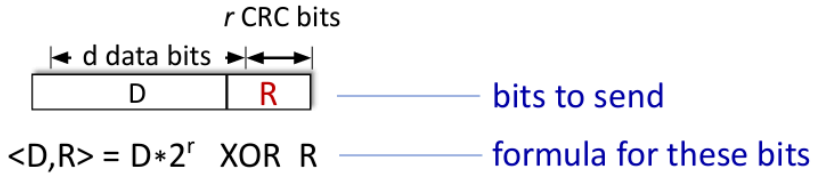
- Sent Message: `101110011`
- Receiver divides by `1001` → remainder 0 if no error.

Bitwise XOR: 0 if both bits are same and 1 if they are different.

```
  1011
⊕ 0101
--------
  1110
```

Cyclic Redundancy Check (CRC)

- more powerful error-detection coding
- D: data bits (given, think of these as a binary number)
- G: bit pattern (generator), of *r+1* bits (given, specified in CRC standard)



sender: compute *r* CRC bits, R, such that <D,R> *exactly* divisible by G (mod 2)
- receiver knows G, divides <D,R> by G. If non-zero remainder: error detected!
- can detect all burst errors less than r+1 bits, why?
- widely used in practice (Ethernet, 802.11 WiFi)

Cyclic Redundancy Check (CRC): example

Sender wants to compute R such that:

$D \cdot 2^r$ XOR $R = nG$

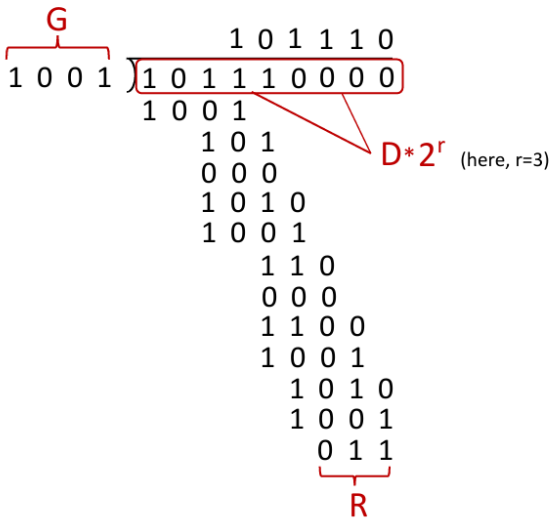
... or equivalently (XOR R both sides):

$D \cdot 2^r = nG$ XOR R

... which says:

if we divide $D \cdot 2^r$ by G, we want remainder R to satisfy:

$R = \text{remainder}\left[\frac{D \cdot 2^r}{G}\right]$ *algorithm for computing R*



* Can you detect all two bits and one bit errors? why?

QUIZ QUESTION

1. For the generator G (= 1001), answer the following questions.

   a) Why can it detect any single bit error in data D?

   Solution:

   Because if any single bit is flipped the remainder will not be zero anymore

   b) Can the above G detect any odd number of bit errors? Why?

   Solution:

   Yes, the key insight here is that G can be divided by 11 (3) as $(x^3+1) = (x+1)(x^2-x+1)$, but any number of odd-number of 1s cannot be divided by 11. Thus, a sequence (not necessarily contiguous) of odd-number bit errors cannot be divided by 11, thus it cannot be divided by G.

Perfect, let's carefully break down both questions and answers so you understand **why the generator G = 1001 works this way**.

Q (a): Why can it detect any single-bit error in data D?

Key Idea:

- CRC detects errors by dividing the received bit sequence by the generator polynomial **G**.
- If the sequence is divisible by G → no error detected.
- If not divisible → error detected.

Single-Bit Error:

- A **single-bit error** means exactly one bit in the transmitted frame flips (0 → 1 or 1 → 0).
- Mathematically, a single-bit error corresponds to an error polynomial of the form:

E(x)=xkE(x) = x^k

(where k is the position of the flipped bit).

- Now, for CRC to **miss the error**, the error polynomial must be divisible by **G**.

Why Detection Works:

- Our generator is:
  G(x)=x3+1=1001G(x) = x^3 + 1 = 1001
- This polynomial **cannot divide** a single power of xkx^k.
- Therefore, if a single bit is flipped, the division remainder will **never be 0**.
- That means **all single-bit errors are guaranteed to be detected**.

✅ This matches the short solution: *"Because if any single bit is flipped the remainder will not be zero anymore."*

Q (b): Can the above G detect any odd number of bit errors? Why?

Step 1: Factorizing G

- Our generator:
  G(x)=x3+1G(x) = x^3 + 1
- Factorizing:
  x3+1=(x+1)(x2−x+1)x^3 + 1 = (x + 1)(x^2 - x + 1)

This shows **G has (x + 1) as a factor**.

Step 2: What does (x + 1) mean?

- The polynomial **(x + 1)** has a special property:
    - If **(x + 1)** divides an error pattern, that pattern must contain an **even number of 1s**.
- Why? Because when you evaluate any binary polynomial at x = 1:
    - If the number of 1s is **even**, the polynomial evaluates to 0 (divisible by x+1).
    - If the number of 1s is **odd**, the polynomial evaluates to 1 (not divisible).

Step 3: Odd number of errors

- An error pattern with an **odd number of bit flips** produces a polynomial with an odd number of 1s.
- Since **(x + 1)** cannot divide a polynomial with an odd number of 1s,
  G(x) cannot divide it either.G(x) \text{ cannot divide it either.}
- Thus, any error with an **odd number of flipped bits** is guaranteed to be detected.

✅ This matches the solution:

- *“G can be divided by (x+1)... but any odd number of 1s cannot be divided by (x+1). Thus odd-number bit errors cannot be divided by G and therefore cannot escape detection.”*

Final Summary

- **Single-bit errors:** Always detected, because G cannot divide a single power of x.
- **Odd-number errors:** Always detected, because G contains (x + 1), which forces even parity.
- **Even-number errors (≥ 2):** Some of these may slip through undetected (depending on pattern).

👉 Would you like me to also show you a **worked-out numerical example** (say, flipping 1 bit vs 3 bits in a CRC calculation with G=1001) to see this rule in action?

6.3 Multiple Access Links and Protocols

In computer networks, links can be broadly divided into **point-to-point links** and **broadcast links**:

- **Point-to-point link:**
    - Connects a **single sender** and a **single receiver**.
    - Examples: PPP (Point-to-Point Protocol), HDLC (High-Level Data Link Control).
- **Broadcast link:**
    - A **shared medium** where multiple nodes (senders and receivers) are connected.
    - When one node transmits, the frame is broadcast and received by all others.
    - Examples: Ethernet, WiFi.

The Multiple Access Problem

In a broadcast link:

- Multiple nodes may attempt to transmit at the same time.
- This causes **collisions** → signals overlap, making all frames unreadable.
- Collided frames are lost, and bandwidth is wasted.

If many nodes are active simultaneously, frequent collisions drastically reduce channel efficiency.

Thus, a method is needed to **coordinate transmissions** among nodes. This is the role of **multiple access protocols**.

Human Analogy

The situation is similar to communication in a **cocktail party** or **classroom**:

- Many people share the same medium (air).
- People must follow social “protocols” to avoid chaos, such as:
    - Give everyone a chance to speak.
    - Don’t interrupt.
    - Raise your hand before asking a question.
    - Don’t monopolize the conversation.

Similarly, computer networks rely on **multiple access protocols** to regulate transmissions.

Types of Multiple Access Protocols

Nearly all multiple access protocols fall into three main categories:

1. **Channel Partitioning Protocols** – Divide the channel into fixed portions (time slots, frequency bands, codes).

2. **Random Access Protocols** – Allow nodes to transmit at will, with retransmission strategies after collisions.

3. **Taking-Turns Protocols** – Nodes take turns accessing the channel in an orderly fashion.

These will be explored in detail later.

Characteristics of an Ideal Multiple Access Protocol

For a channel with rate **R bps**, a good multiple access protocol should satisfy:

1. **Full efficiency for single node:**
    - If only one node has data, it should transmit at full rate (R bps).
2. **Fair share when multiple nodes are active:**
    - If **M nodes** have data, each should get an average rate of **R/M bps**.
    - This does not mean instantaneously equal rates, but fairness over time.
3. **Decentralization:**
    - Should not rely on a single master node (avoids a single point of failure).
4. **Simplicity:**
    - Easy and inexpensive to implement.

6.3.1 Channel Partitioning Protocols

Channel partitioning protocols **divide the shared broadcast channel** into fixed portions so that each node gets a fair and collision-free share of the bandwidth. The three main techniques are **TDM, FDM, and CDMA**.

---

1. Time-Division Multiplexing (TDM)

- **How it works:**
    - The channel time is divided into **frames**, and each frame is further divided into **N slots** (one per node).
    - Each node is assigned a fixed time slot in every frame.
    - When a node has data, it sends during its slot; if it has no data, the slot goes unused.
    - Typically, a slot is sized to hold **one packet**.
- **Analogy:**
    - In a cocktail party, each person gets a **turn to speak for a fixed period**, then the next person speaks, and so on in a cycle.
- **Advantages:**
    - No collisions (perfectly coordinated).
    - Fair sharing: each node gets **R/N bps**.
- **Disadvantages:**
    - If only one node has data, it is still limited to **R/N bps**.
    - Nodes must wait for their turn even if they are the only ones with packets to send.

---

2. Frequency-Division Multiplexing (FDM)

- **How it works:**
    - The channel bandwidth **R bps** is divided into **N smaller frequency bands**, each of width **R/N**.
    - Each node is assigned one frequency band for transmission.
- **Analogy:**
    - Like radio stations, where each station broadcasts on a **different frequency** without interfering.
- **Advantages:**
    - No collisions.
    - Fair sharing of bandwidth among all nodes.
- **Disadvantages:**
    - Each node is limited to **R/N bps**, even if it is the only active sender.
    - Spectrum is wasted if some nodes have no data.

---

3. Code Division Multiple Access (CDMA)

- **How it works:**
    - Instead of dividing time or frequency, each node is assigned a **unique code**.
    - Data bits are encoded using that code before transmission.
    - With properly chosen codes, multiple nodes can transmit **simultaneously** over the same channel, and receivers can still decode the intended message if they know the sender's code.
- **Advantages:**
    - Multiple users can share the channel **at the same time**.

    - Very resistant to interference and jamming.
    - Widely used in **wireless systems** (cellular telephony, military communication).
- **Disadvantages:**
    - Requires careful code design.
    - More complex than TDM and FDM.

Summary

- **TDM**: Share by **time slots** (turn-taking).
- **FDM**: Share by **frequency bands**.
- **CDMA**: Share by **unique codes**, allowing simultaneous transmissions.

All three eliminate collisions and fairly allocate bandwidth, but they **waste capacity** when only a few nodes are active.

6.3.2 Random Access Protocols

In **random access protocols**, nodes do not have pre-allocated slots or frequencies. Instead, each node transmits **at the full channel rate (R bps)** whenever it has data. The challenge is to resolve **collisions** when multiple nodes transmit simultaneously.

The general idea:

- If a collision occurs, the involved nodes wait a **random delay** before retransmitting.
- Each node chooses its delay **independently**, so with luck one will transmit while others remain silent, making the transmission successful.

This randomness avoids repeated collisions and keeps the system decentralized.

Slotted ALOHA

One of the earliest and simplest random access protocols.

Assumptions

- All frames are of equal size (L bits).
- Time is divided into **slots** of length L/R seconds (enough to send one frame).
- Nodes can only transmit at the **start of a slot** (synchronized).
- Collisions are detected before the slot ends.

Protocol Rules

1. If a node has a frame to send → it waits for the **next slot** and transmits.
2. If only one node transmits in that slot → **success**.
3. If multiple nodes transmit in the same slot → **collision**, and all frames are lost.
4. Each node involved in a collision will try again in future slots, but only with **probability p**.
    - With probability p → retransmit.
    - With probability (1 − p) → wait and try in a later slot.

This probability prevents all nodes from retransmitting immediately and colliding again.

Advantages of Slotted ALOHA

- A node can use the **full channel rate R** if it is the only active node.
- **Decentralized**: no master controller, nodes decide independently.

- **Simple** to implement.

Disadvantages / Inefficiency

- Collisions waste slots.
- Empty slots occur when all nodes defer transmission.
- Only slots with **exactly one transmission** are useful.

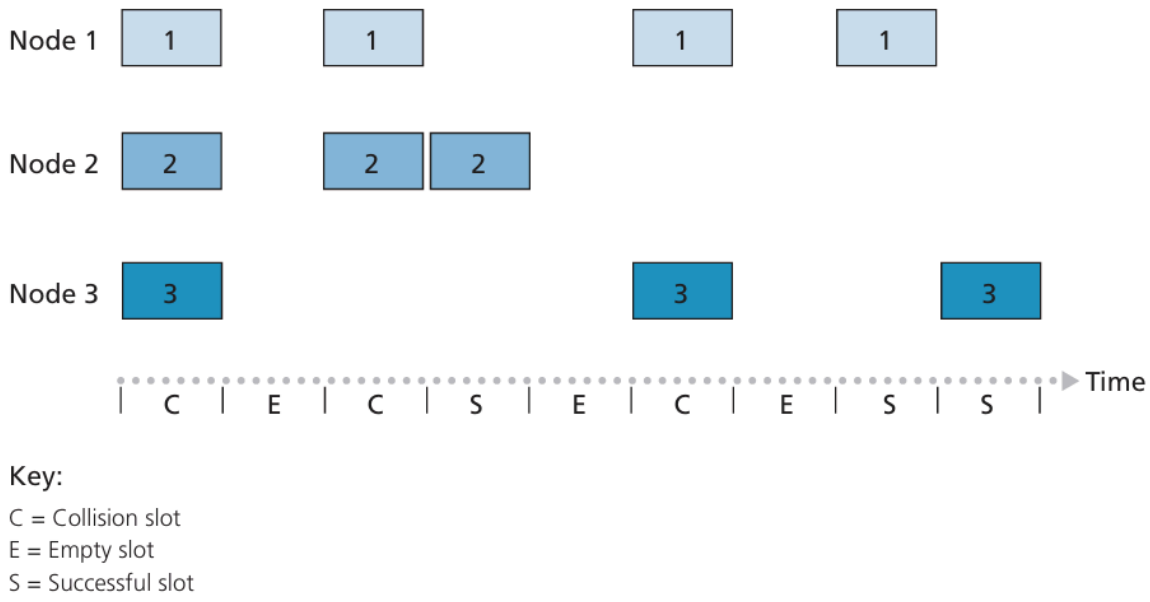


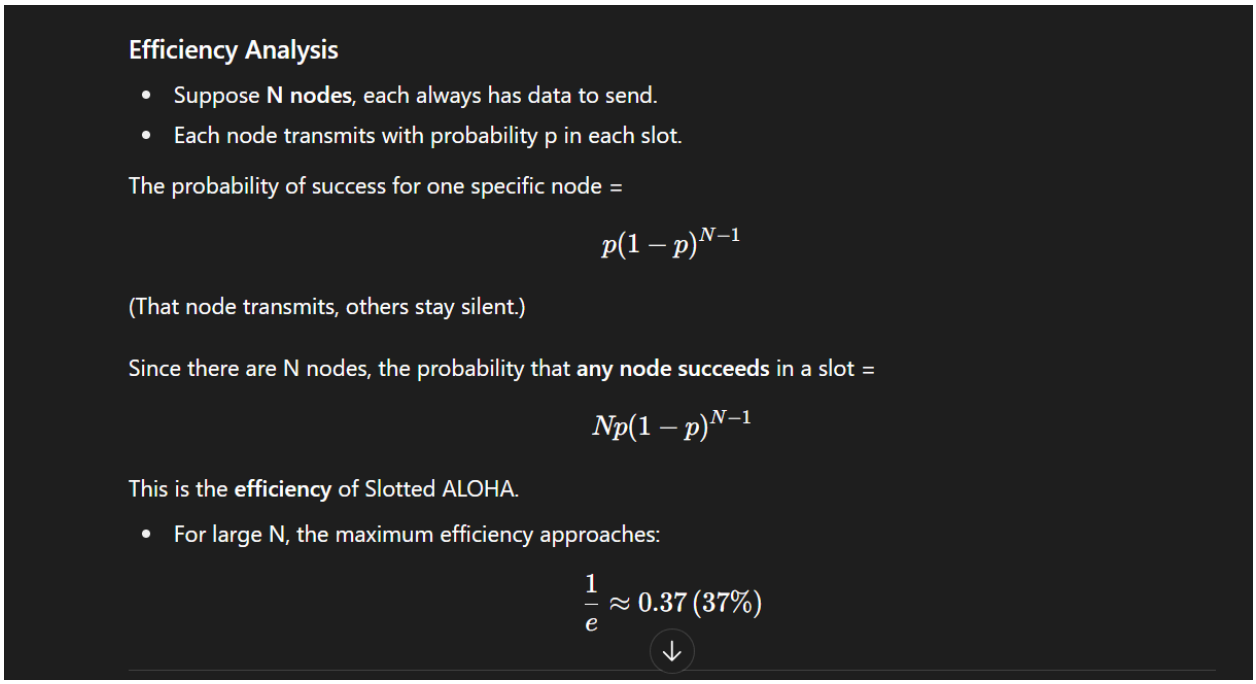
Figure 6.10 ♦ Nodes 1, 2, and 3 collide in the first slot. Node 2 finally succeeds in the fourth slot, node 1 in the eighth slot, and node 3 in the ninth slot

Figure 6.10 Example:

- Slot 1 → Nodes 1, 2, and 3 all transmit → **collision (C)**.
- Slot 2 → No one transmits → **empty (E)**.
- Slot 4 → Only Node 2 transmits → **success (S)**.

- Slot 8 → Node 1 transmits → **success (S)**.
- Slot 9 → Node 3 transmits → **success (S)**.

So, useful slots = those where only one node succeeds.



Key Insight

- At best, only **37% of the channel capacity** is used for successful transmissions.
- The rest is wasted:
    - ~37% slots are empty.
    - ~26% slots have collisions.
- Example: A 100 Mbps channel effectively delivers **< 37 Mbps** under heavy load.

👉 In summary, **Slotted ALOHA** is simple and decentralized, but only achieves **37% maximum efficiency** in large-scale usage due to wasted slots from collisions and idle times.

Would you like me to also explain the **Unslotted (Pure) ALOHA** version and compare its efficiency with Slotted ALOHA?

Carrier Sense Multiple Access (CSMA)

ALOHA protocols waste bandwidth because nodes transmit **without checking** whether others are already transmitting. **Carrier Sense Multiple Access (CSMA)** improves efficiency by adding two simple but powerful rules inspired by human conversation:

Two Core Rules of CSMA

1. **Carrier Sensing (Listen before speaking):**
    - Before transmitting, a node **listens to the channel**.
    - If another frame is being transmitted, it waits until the channel is idle.
    - Only then does it begin transmission.
2. **Collision Detection (Stop if someone else speaks at the same time):**
    - While transmitting, a node **monitors the channel**.
    - If it detects interference (another node also transmitting), it **immediately stops** transmitting.
    - It then waits a random time before retrying.
    - This mechanism is called **CSMA/CD** (Carrier Sense Multiple Access with Collision Detection), famously used in classic Ethernet.

These rules reduce collisions and wasted bandwidth compared to ALOHA.

Why Collisions Still Occur

Even with CSMA, collisions are **not fully avoidable**. The reason lies in **propagation delay**:

- A signal takes time to travel across the medium (though very fast, it's not instantaneous).
- A node may sense the channel as **idle** because the signal from another node hasn't reached it yet.
- Thus, two nodes may start transmitting almost simultaneously → collision occurs.

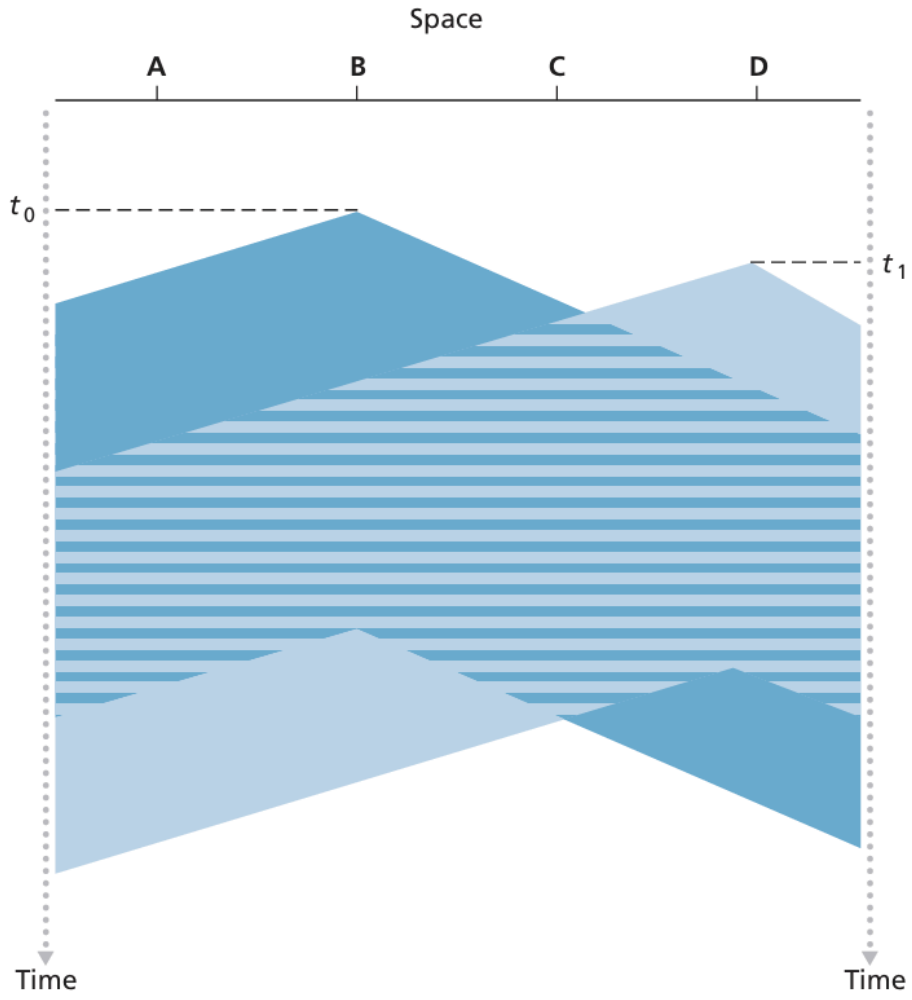


Figure 6.12 ♦ Space-time diagram of two CSMA nodes with colliding transmissions

---

Space-Time Diagram Explanation (Figure 6.12)

The figure illustrates this scenario with four nodes (A, B, C, D) on a shared bus.

- At **time t_0**:
    - Node **B** senses the channel idle.
    - It begins transmitting.
    - Its bits propagate outward from B in both directions (towards A and towards D).
- At **time $t_1 > t_0$**:
    - Node **D** also has a frame to send.
    - At that moment, B's bits have **not yet reached D** (due to propagation delay).
    - So D mistakenly believes the channel is idle and starts transmitting.
- A short time later:
    - B's signal reaches D, and D's signal reaches B.
    - Their transmissions interfere → **collision**.

Key Insight

- Even with carrier sensing, **propagation delay** makes it possible for two nodes to think the channel is free and transmit simultaneously.
- The **longer the propagation delay**, the higher the risk of such collisions.
- Hence, CSMA efficiency strongly depends on the ratio:

Propagation delayFrame transmission time\frac{\text{Propagation delay}}{\text{Frame transmission time}}

The smaller this ratio, the fewer collisions.

👉 In summary, **CSMA improves over ALOHA** by reducing wasted slots through carrier sensing and collision detection. However, due to **propagation delays**, collisions cannot be completely eliminated, only minimized.

Carrier Sense Multiple Access with Collision Detection (CSMA/CD)

CSMA/CD is an improvement over CSMA that not only listens before transmitting but also **detects and reacts to collisions while transmitting**. This avoids wasting the channel on full-frame collisions.

Problem with CSMA Alone (Figure 6.12 Recap)

- In plain CSMA, if two nodes (say B and D) start transmitting almost simultaneously, they **collide**.
- However, without collision detection, both nodes continue sending their entire frames, which are already corrupted.
- This wastes bandwidth and time.

How CSMA/CD Works (Figure 6.13)

Step-by-Step Operation

1. **Frame Preparation:**
    - Adapter receives a datagram from the network layer, makes a link-layer frame, and puts it in the buffer.
2. **Carrier Sense:**
    - If the channel is idle → begin transmitting immediately.
    - If the channel is busy → wait until it becomes idle, then transmit.

3. **Collision Detection During Transmission:**
    - While transmitting, the node keeps monitoring the channel for **energy from other nodes**.
    - If no other energy is detected → transmission finishes successfully.
    - If another node transmits at the same time → signals interfere → **collision detected**.
4. **Abort Transmission:**
    - As soon as a collision is detected, the node stops sending the frame.
    - This prevents wasting time sending the entire corrupted frame.
5. **Random Backoff:**
    - After aborting, the node waits a **random amount of time** before retrying.
    - Then it repeats from Step 2.

Why Random Backoff?

- If all colliding nodes restarted at the same fixed time, they would **collide again and again**.
- Randomness ensures fairness and reduces repeat collisions.

Binary Exponential Backoff Algorithm

Used in Ethernet and DOCSIS cable networks:

- After **n collisions**, the node chooses a random number **K** from:
  {0,1,2,...,2n−1}\{0, 1, 2, ..., 2^n - 1\}
- The node waits **K × 512 bit times** before retransmitting.

    - (In 100 Mbps Ethernet, 512 bit times = 5.12 microseconds.)

- Examples:
    - 1st collision → K from {0, 1}.
    - 2nd collision → K from {0, 1, 2, 3}.
    - 3rd collision → K from {0 … 7}.
    - After 10+ collisions → K from {0 … 1023}.
- The wait interval **grows exponentially** with the number of collisions, giving the network time to “cool off.”

Important Note

- Every **new frame** starts fresh. Past collisions don’t affect it.
- A node may succeed immediately with a new frame even if other nodes are in backoff.

Advantages of CSMA/CD

- Prevents wasting bandwidth on fully collided frames.
- Efficient under light and moderate loads.
- Used successfully in **classic Ethernet (wired LANs)**.

👉 In summary, **CSMA/CD** improves performance over CSMA by **detecting collisions early and aborting transmission**, combined with **binary exponential backoff** to resolve repeated collisions fairly.

Would you like me to also show you a **numerical timeline example** (two nodes colliding, detecting, aborting, and retrying with backoff) to make the process crystal clear?

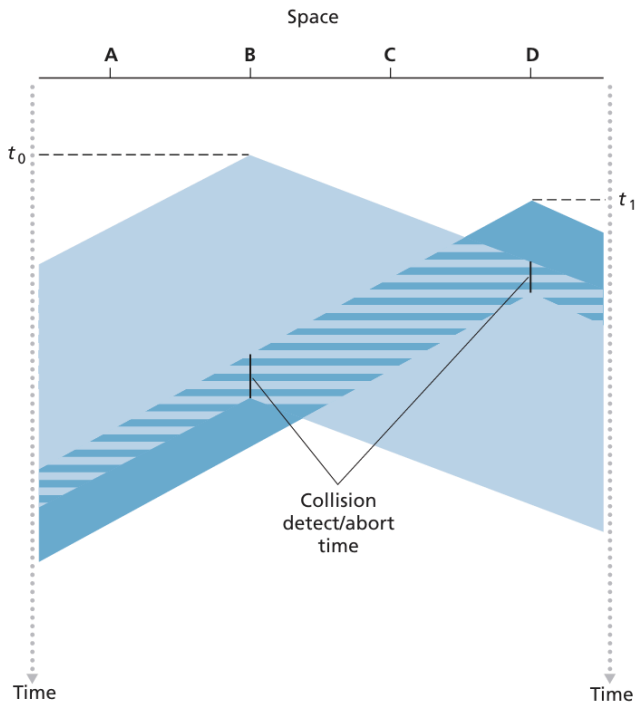


Figure 6.13 ♦ CSMA with collision detection

Got it 👍 Let's carefully explain **what is happening in Figure 6.13 (CSMA with Collision Detection).**

What the Figure Shows

- **Horizontal axis = Space** → locations of nodes A, B, C, D along the broadcast channel.
- **Vertical axis = Time** → how signals propagate as time goes forward.
- **Blue diagonal bands** = signals (frames) traveling across the shared medium.
- **Shaded overlap** = collision area.

Step-by-Step Explanation

1. **At time t0t_0:**
    - Node **B** senses the channel is idle → starts transmitting.

- Its signal (frame bits) begins to propagate outward in both directions (towards A and towards D).

2. **At time t1>t0t_1 > t_0:**
    - Node **D** also senses the channel.
    - Since B's signal has **not yet reached D** (propagation delay), D mistakenly thinks the channel is free and starts transmitting.
3. **Collision Happens:**
    - As B's signal reaches D and D's signal reaches B, they **overlap in the medium**.
    - This overlap is the **collision region** shown in the figure.
    - Both frames are corrupted.
4. **Collision Detection:**
    - Unlike plain CSMA (Figure 6.12), both B and D **monitor the channel while transmitting**.
    - They notice unexpected energy on the medium → realize a collision is happening.
5. **Abort Transmission:**
    - As soon as they detect the collision, both B and D **stop transmitting immediately**.
    - This is marked in the figure as **"Collision detect/abort time."**
    - This saves bandwidth: instead of sending entire corrupted frames, they cut off transmission early.

Key Insight from the Figure

- Collisions **still occur** due to propagation delay (same as in CSMA).

- The **difference** with CSMA/CD:
    - In CSMA → nodes keep transmitting full frames, wasting time.
    - In CSMA/CD → nodes **detect quickly and abort**, minimizing wasted channel time.

✅ So, **Figure 6.13 demonstrates two nodes (B and D) colliding, detecting the collision, and aborting early**—which is the essential improvement of CSMA/CD over plain CSMA.

Would you like me to also **sketch a small timeline example** (showing how Node B and Node D retry after applying the binary exponential backoff)?